

On the Weaknesses of Protective Mechanisms in LLMs

Nizaul Rahman [2025MCS2099] Ansh Gupta [2025MCS2015]

February 13, 2026

1. Paper Details

Title:

[MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots.](#)

Authors:

Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu.

Citations: 302

Published Date: 25 October 2023

Conference: Network and Distributed System Security (NDSS) Symposium

2. Project Title

Automating LLM Jailbreaking: Reverse-Engineering and Generative Attack Frameworks

3. Problem Statement

Commercial Large Language Model (LLM) chatbots are increasingly vulnerable to jailbreak attacks, where malicious actors manipulate prompts to bypass safety alignments and generate content that violates usage policies. Despite the prevalence of these systems, the "black-box" nature of commercial defences—such as those guarding Google Bard and Bing

Chat—prevents security researchers from understanding the efficacy of modern protection layers.

Traditional jailbreak prompts, largely designed for ChatGPT (GPT-3.5), exhibit a catastrophic drop in efficacy when applied to more rigorously guarded models. This indicates that service providers have implemented undisclosed, dynamic defence mechanisms—including real-time content moderation and keyword filtering—that go beyond simple input screening. As shown in the comparison below, existing static attacks fail to penetrate the more sophisticated defensive perimeters of contemporary chatbots.

Effectiveness Comparison of Existing Jailbreaks

Model	Average Success Rate (%)
GPT-3.5	21.12%
GPT-4	7.13%
Google Bard	0.40%
Bing Chat	0.63%

This lack of transparency is compounded by the fact that most service providers provide non-informative, generic refusals (e.g., "I cannot help with that") rather than identifying specific policy infractions. This necessitated a reverse-engineering approach to uncover how these "black-box" systems actually process and reject malicious queries.

5. Proposed Solutions and Extensions

5.1 Time-Based Reverse Engineering

To bypass the lack of direct feedback from commercial chatbots, this project utilizes "Time-based LLM Testing." This methodology is directly inspired by **time-based blind SQL injection** attacks, where internal database logic is inferred by observing the time delay in responses. In the framework, response time serves as a crucial side-channel indicator of the internal generation and moderation status.

Our technical foundation rests on the fact that LLMs generate text token-by-token. As established in the foundational research, there is a statistically significant positive linear correlation between token count and generation time. The paper utilize the **Pearson**

correlation coefficient (r) to validate this relationship, noting values such as $r = 0.838$ ($p < 0.001$) for GPT-4 and $r = 0.762$ ($p < 0.001$) for Bard, proving that response time is a reliable proxy for the duration of the internal generation process.

By manipulating the placement of malicious content within prompts and measuring the resulting time delta, we achieved the following insights:

- **Defence Phase:** Statistical analysis shows that Bard and Bing Chat do not implement significant input-filtering. Instead, the jailbreak prevention schemes conduct checks on the model-generated results.
- **Real-Time Dynamics:** Z-test results (average z-score of 29.48, $p < 0.01$) indicate that these services detect violations during the generation process in real-time, rather than performing a single post-generation check.
- **Mechanism Characterisation:** Testing with "red-flag" keyword insertion suggests that both Bard and Bing Chat employ a hybrid approach involving both **keyword matching** and **semantic analysis** to terminate policy-violating streams instantly.

5.2 Automated Jailbreak Generation

The MASTERKEY framework automates the discovery of vulnerabilities via a three-stage generative pipeline:

1. **Dataset Building and Augmentation:** They expand an initial set of 85 jailbreak prompts through self-instruction and text-style transfer. By using a "teacher" model to rephrase patterns into general expressions (e.g., replacing "ChatGPT" with "you"), we create a diverse library of universal templates.
2. **Continuous Pre-training and Task Tuning:** We fine-tune an open-source Vicuna-13b model to internalise these jailbreaking patterns, sharpening its ability to generate adversarial text-style transfers.
3. **Reward Ranked Fine Tuning (RAFT):** To optimize the generator, we implement a reinforcement learning strategy. Prompts are ranked based on their ability to bypass multiple disparate chatbots. The objective function is defined as: **Reward = $\sum_{i=1}^n \text{JailbreakSuccess}_i$** where n represents the total number of target LLMs, and successful bypasses are rewarded linearly to encourage the generation of "model-agnostic" attack patterns.

5.3 Proposed Solutions:

1. **Input Filtering:** ChatBots should process input before generating any response. User intent should be detected using Deep Learning Models from the input itself halting the generation of response.
2. **Keyword Flagging in Input:** ChatBots should immediately block the question if some flagged keyword appears in it.

5.4 Proposed Extension: Cross-Language Robustness Testing

We propose extending this research to assess cross-language compatibility, specifically targeting non-English models like Baidu's Ernie. This involves translating successful automated jailbreak patterns into simplified Chinese to test the generalisability of attack patterns across different linguistic and regulatory frameworks.

Implementation Risks: We identify significant "real-world" constraints for this extension, including Baidu's aggressive **account suspension policies** and stringent **rate limits** triggered by repeated unsuccessful jailbreak attempts. Our methodology will incorporate a distributed testing approach to mitigate these risks.

6. Evaluation Metrics

We utilise two primary metrics to quantify attack efficacy:

- **Query Success Rate (QSR):** Defined as the ratio of successful jailbreak queries (S) to the total number of queries (T), calculated as $Q = S / T$.
- **Prompt Success Rate (PSR):** Defined as the proportion of generated jailbreak prompts (G) that result in at least one successful query across the total pool of generated prompts (P), calculated as $J = G / P$.

7. Key Takeaways

1. **Defence Insights:** Dynamic, real-time moderation is the industry standard for guarded models. Reverse-engineering reveals a reliance on hybrid keyword-mapping and semantic-analysis defences rather than simple input sanitisation.
2. **Attack Success:** MASTERKEY achieved an average QSR of 21.58%, a nearly 3x improvement over the 7.33% average success rate of previous manual methods, effectively breaching Bard and Bing Chat for the first time.
3. **Transparency Gaps:** Service providers lack informative feedback loops, necessitating side-channel analysis (time-based blind SQL injection analogues) to understand defensive postures.
4. **Generalisability:** The framework produces **model-agnostic** prompts that are effective across a diverse range of architectures (GPT-4, Bard, Bing, Ernie), proving that automated LLM-based attacks can consistently outpace static, "black-box" security implementations.